

Chapitre 0 : Fondamentaux du C

Syntaxe Complète et Subtilités

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

1 Types et Représentation

1.1 Types Entiers

Le C fournit plusieurs types entiers de tailles différentes. La taille exacte dépend de l'architecture.

Types entiers signés

`signed char` (au moins 8 bits), `short` (au moins 16 bits), `int` (au moins 16 bits, généralement 32), `long` (au moins 32 bits), `long long` (au moins 64 bits).

Types entiers non signés

Préfixe `unsigned` : `unsigned char`, `unsigned short`, `unsigned int`, `unsigned long`, `unsigned long long`.

Type	Taille (x86-64)	Min/Max
<code>char</code>	1 byte	-128 à 127
<code>short</code>	2 bytes	-32768 à 32767
<code>int</code>	4 bytes	-2^{31} à $2^{31} - 1$
<code>long</code>	8 bytes	-2^{63} à $2^{63} - 1$
<code>long long</code>	8 bytes	-2^{63} à $2^{63} - 1$

Garanties du standard

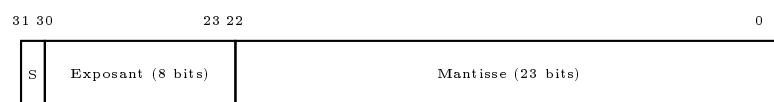
Le standard C garantit uniquement : $\text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$. Sur certaines architectures 32 bits, `long` = 4 bytes.

1.2 Types Flottants

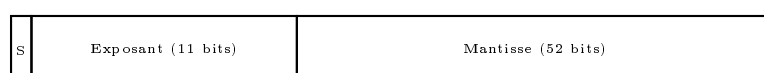
Types à virgule flottante

`float` (généralement 32 bits IEEE 754 simple précision), `double` (généralement 64 bits IEEE 754 double précision), `long double` (80 ou 128 bits selon architecture).

float (32 bits)



double (64 bits)



1.3 Type char et Signedness

Ambiguïté du type char

Le standard C ne spécifie PAS si `char` est signé ou non signé. Sur x86-64 (gcc), `char` est signé par défaut. Sur ARM, il est souvent non signé. Utiliser explicitement `signed char` ou `unsigned char` pour la portabilité.

Listing 1: Comportement dépendant de l'implémentation

```
1 char c = 200; // Comportement non portable
2 // x86-64 gcc: c = -56 (dépassement en signé)
3 // ARM gcc: c = 200 (non signé)
4
5 // Portable:
6 unsigned char uc = 200; // Toujours 200
7 signed char sc = 200; // Overflow défini: wrap around
```

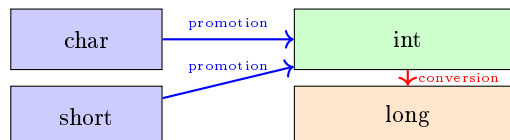
1.4 Conversions Implicites

Promotion entière

Les types plus petits que `int` sont automatiquement promus à `int` dans les expressions. Ceci s'appelle la "promotion entière intégrale".

Listing 2: Promotion et conversions

```
1 short a = 10, b = 20;
2 int c = a + b; // a et b promus en int avant addition
3
4 unsigned char uc = 255;
5 int i = uc + 1; // uc promu en int, résultat = 256
6
7 // Conversion signé/non-signé (dangereux)
8 int si = -1;
9 unsigned int ui = 1;
10 if (si < ui) // FAUX! si converti en unsigned: 0xFFFFFFFF > 1
11     printf("si < ui\n");
12 else
13     printf("si >= ui\n"); // S'exécute!
```



1.5 Opérateur sizeof

sizeof

Opérateur (pas une fonction) qui retourne la taille en bytes d'un type ou d'une expression. Évalué à la compilation. Retourne un `size_t` (entier non signé).

Listing 3: Utilisation de sizeof

```
1 // Sur des types
2 printf("%zu\n", sizeof(int)); // 4 (x86-64)
3 printf("%zu\n", sizeof(char)); // 1 (toujours)
4 printf("%zu\n", sizeof(double)); // 8
5
6 // Sur des variables (parenthèses optionnelles)
7 int x = 42;
8 printf("%zu\n", sizeof(x)); // 4
9 printf("%zu\n", sizeof x); // 4 (valide sans parenthèses)
10
11 // Sur des expressions (non évaluées!)
12 int a = 5;
13 printf("%zu\n", sizeof(a++)); // 4, et a reste 5!
```

sizeof et expressions

L'expression dans `sizeof` n'est PAS évaluée. Seul le type est déterminé à la compilation. Donc `sizeof(a++)` ne modifie pas `a`.

Listing 4: sizeof et tableaux vs pointeurs

```
1 int arr[10];
2 int *ptr = arr;
3
4 printf("%zu\n", sizeof(arr)); // 40 (10 * 4 bytes)
5 printf("%zu\n", sizeof(ptr)); // 8 (taille d'un pointeur x86-64)
6
7 // Nombre d'éléments d'un tableau
8 size_t n = sizeof(arr) / sizeof(arr[0]); // 10
```

2 Variables et Classes de Stockage

2.1 Déclaration, Définition et Initialisation

Déclaration vs Définition

Déclaration : introduit un identifiant et son type, sans allouer de mémoire.

Définition : déclaration + allocation mémoire. Toute définition est une déclaration.

Listing 5: Déclaration et définition de variables

```
1 // Définition (déclaration + allocation)
2 int x; // Définition, valeur indéterminée
3 int y = 42; // Définition + initialisation
4
5 // Déclaration pure (pas d'allocation ici)
6 extern int z; // Déclaré ici, défini ailleurs
7
8 // Pour les fonctions
9 int add(int, int); // Déclaration (prototype)
10 int add(int a, int b) { return a + b; } // Définition
```

Variables non initialisées

Une variable locale non initialisée a une valeur **indéterminée**. La lire est un comportement indéfini (UB). Les variables globales et `static` sont initialisées à zéro par défaut.

Listing 6: Initialisation

```
1 int a; // Global: initialisé à 0
2 static int b; // Static: initialisé à 0
3
4 int main() {
5     int c; // Local: valeur INDÉTERMINÉE (UB si lu)
6     int d = 0; // Local: initialisé explicitement
7
8     static int e; // Static local: initialisé à 0
9
10    printf("%d\n", c); // UB! Valeur imprévisible
11    return 0;
12 }
```

2.2 Portée et Durée de Vie

Portée (scope)

Portée de bloc : variable visible du point de déclaration jusqu'à la fin du bloc `{}`.

Portée de fichier : variable déclarée hors de toute fonction, visible dans tout le fichier.

Listing 7: Portée et masquage

```
1 int x = 10; // Portée fichier (globale)
2
3 void foo() {
4     int x = 20; // Masque le x global
5     printf("%d\n", x); // 20
```

```

6
7
8     {
9         int x = 30; // Masque le x de foo
10        printf("%d\n", x); // 30
11    }
12
13    printf("%d\n", x); // 20 (x de foo)
14}
15
16int main() {
17    printf("%d\n", x); // 10 (x global)
18    foo();
19    return 0;
20}

```

Durée de vie

Automatique : durée du bloc (variables locales).

Statique : durée du programme (globales et `static`).

2.3 Classes de Stockage

auto (implicite)

Classe par défaut pour les variables locales. Allocation sur la pile, durée de vie = bloc englobant.

Listing 8: auto (rarement écrit explicitement)

```

1 void foo() {
2     auto int x = 5; // Équivalent à: int x = 5;
3     // auto est implicite pour les locales
4 }

```

static

Variable locale static : durée de vie = programme, portée = bloc.

Variable globale static : portée limitée au fichier (liaison interne).

Fonction static : visible uniquement dans le fichier.

Listing 9: static local : persistance entre appels

```

1 void counter() {
2     static int count = 0; // Initialisé UNE seule fois
3     count++;
4     printf("Appel #%d\n", count);
5 }
6
7 int main() {
8     counter(); // Appel #1
9     counter(); // Appel #2
10    counter(); // Appel #3
11    return 0;
12}

```

Listing 10: static global : encapsulation au fichier

```

1 // fichier: module.c
2 static int internal_counter = 0; // Invisible hors de ce fichier
3 static void helper() { }         // Fonction interne
4
5 void public_api() { // Visible partout
6     internal_counter++;
7     helper();
8 }

```

extern

Déclare une variable ou fonction définie dans un autre fichier (liaison externe). Pas d'allocation, juste une référence.

Listing 11: extern : exemple multi-fichiers

```

1 // === config.h ===
2 #ifndef CONFIG_H
3 #define CONFIG_H

```

```

4  extern int global_count;    // Déclaration
5  extern void increment();    // Déclaration (extern implicite)
6  #endif
7
8  // === config.c ===
9  #include "config.h"
10 int global_count = 0;       // Définition (une seule fois)
11
12 void increment() {          // Définition
13     global_count++;
14 }
15
16 // === main.c ===
17 #include <stdio.h>
18 #include "config.h"
19
20 int main() {
21     printf("%d\n", global_count); // 0
22     increment();
23     increment();
24     printf("%d\n", global_count); // 2
25     return 0;
26 }
27
28 // Compilation: gcc -o prog main.c config.c

```

Erreurs courantes avec extern

- Oublier la définition : erreur de linkage "undefined reference"
- Définir plusieurs fois : erreur "multiple definition"
- Initialiser dans la déclaration extern : devient une définition!

Listing 12: Erreurs avec extern

```

1  // ERREUR: extern avec initialisation = définition
2  extern int x = 5;    // C'est une DÉFINITION, pas une déclaration
3
4  // CORRECT
5  extern int x;        // Déclaration
6  int x = 5;           // Définition (dans un autre fichier ou plus bas)

```

2.4 Constantes : const vs #define

const

Qualificateur de type. La variable est en lecture seule après initialisation. Typée, scopée, visible au débogueur.

#define

Directive préprocesseur. Substitution textuelle avant compilation. Pas de type, portée = fin du fichier (ou #undef).

Listing 13: const vs #define

```

1  #define MAX_MACRO 100    // Substitution textuelle
2  const int MAX_CONST = 100; // Variable constante
3
4  // Typage
5  #define PI_MACRO 3.14159
6  const double PI_CONST = 3.14159;
7
8  // Scope
9  void foo() {
10     #define LOCAL_MACRO 42 // Visible PARTOUT après cette ligne!
11     const int LOCAL_CONST = 42; // Visible seulement dans foo
12 }
13
14 // Débogage
15 // gdb peut afficher MAX_CONST, pas MAX_MACRO (disparu après préprocesseur)

```

Listing 14: Pièges du #define

```

1  #define SQUARE(x) x * x
2
3  int a = SQUARE(3);    // 3 * 3 = 9, OK
4  int b = SQUARE(1 + 2); // 1 + 2 * 1 + 2 = 5, FAUX!
5
6  // Solution: parenthèses

```

```

7 #define SQUARE_SAFE(x) ((x) * (x))
8 int c = SQUARE_SAFE(1 + 2); // ((1 + 2) * (1 + 2)) = 9, OK
9
10 // Mais toujours des problèmes
11 int d = SQUARE_SAFE(a++); // ((a++) * (a++)) : a incrémenté 2 fois!

```

Recommandation

Préférer `const` à `#define` pour les constantes. Utiliser `#define` pour la compilation conditionnelle et les macros complexes uniquement.

3 Opérateurs et Évaluation

3.1 Opérateurs Arithmétiques

Opérateurs arithmétiques

`+`, `-`, `*`, `/`, `%` (modulo, entiers seulement).
 Division entière : $7 / 2 = 3$, $-7 / 2 = -3$ (troncature vers zéro en C99).
 Modulo : signe du résultat = signe du dividende.

Listing 15: Division et modulo

```

1 int a = 7 / 2; // 3
2 int b = -7 / 2; // -3 (C99: troncature vers zéro)
3 int c = 7 % 2; // 1
4 int d = -7 % 2; // -1 (signe de -7)
5 int e = 7 % -2; // 1 (signe de 7)
6
7 // Attention au type!
8 float f = 7 / 2; // 3.0 (division entière puis conversion)
9 float g = 7.0 / 2; // 3.5 (division flottante)

```

3.2 Opérateurs Bit à Bit

Opérateurs bit à bit (entiers seulement)

`&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (shift gauche), `>>` (shift droite).

Listing 16: Opérations bit à bit

```

1 unsigned int x = 0x89AB; // 1000 1001 1010 1011
2 unsigned int y = 0x1234; // 0001 0010 0011 0100
3
4 unsigned int a = x & y; // 0x0020 (AND)
5 unsigned int b = x | y; // 0x9BBF (OR)
6 unsigned int c = x ^ y; // 0x9B9F (XOR)
7 unsigned int d = ~x; // 0xFFFF7654 (NOT, 32 bits)
8
9 // Shifts
10 unsigned int e = x << 2; // 0x226AC (shift gauche, * 4)
11 unsigned int f = x >> 2; // 0x226A (shift droite logique, / 4)
12
13 // Shift droite signé vs non-signé
14 int si = -8; // 0xFFFFFFF8
15 int g = si >> 1; // -4 (shift arithmétique: sign extend)
16 unsigned int ui = -8;
17 unsigned int h = ui >> 1; // 0x7FFFFFFC (shift logique: zéro extend)

```

Shift droite et type signé

Le comportement de `>>` sur les types signés est défini par l'implémentation (généralement arithmétique avec extension de signe). Utiliser des types non signés pour des manipulations de bits prévisibles.

3.3 Opérateurs Logiques et Court-Circuit

Opérateurs logiques

`&&` (AND logique), `||` (OR logique), `!` (NOT logique). Évaluation en court-circuit : `&&` s'arrête au premier faux, `||` au premier vrai.

Listing 17: Court-circuit et side-effects

```
1 int a = 0, b = 0;
2
3 // && court-circuite
4 if (a != 0 && b/a > 2) { // b/a non évalué car a == 0
5     printf("OK\n");
6 }
7
8 // Ordre d'évaluation garanti
9 int c = 0;
10 if (++c > 0 && ++c > 1) { // c = 2 après évaluation
11     printf("c = %d\n", c);
12 }
13
14 // Erreur classique: & au lieu de &&
15 if (a != 0 & b/a > 2) { // ERREUR: & évalue les deux côtés!
16     printf("Crash si a == 0\n");
17 }
```

3.4 Opérateurs d'Incrément

Incrément et décrémentation

`++x` (pré-incrément : incrémente puis retourne), `x++` (post-incrément : retourne puis incrémente). Idem pour `--`.

Listing 18: Pré vs post incrément

```
1 int a = 5, b, c;
2
3 b = ++a; // a = 6, b = 6 (pré-incrément)
4 c = a++; // c = 6, a = 7 (post-incrément)
5
6 // Danger: modification multiple dans une expression
7 int x = 5;
8 int y = (x++) + (++x); // COMPORTEMENT INDÉFINI (UB)!
9 // x modifié deux fois sans sequence point
```

3.5 Opérateur Ternaire

Opérateur ternaire

`condition ? valeur_si_vrai : valeur_si_faux`. Seule l'expression sélectionnée est évaluée.

Listing 19: Ternaire et side-effects

```
1 int a = 10, b = 20;
2 int max = (a > b) ? a : b; // max = 20
3
4 // Side-effect dans une seule branche
5 int c = 0;
6 int d = (a > b) ? ++a : ++a; // Seulement ++a exécuté
7 // c = 0, a = 11, d = 11
```

3.6 Précédence et Associativité

Priorité	Opérateurs	Associativité
Haute	() [] -> .	Gauche
	! ~ ++ -- (cast)	Droite
	* / %	Gauche
	+ -	Gauche
	< >	Gauche
	< <= > >=	Gauche
	== !=	Gauche
	&	Gauche
	^	Gauche
		Gauche
	&&	Gauche
		Gauche
	?:	Droite
Basse	= += -= etc.	Droite

Listing 20: Pièges de précédence

```
1 int a = 5, b = 10, c;
2
3 // Priorité inattendue
4 c = a & b == 5; // Équivalent à: a & (b == 5)
5 // NON (a & b) == 5 !
6
7 // Shift et addition
8 c = 1 << 2 + 3; // Équivalent à: 1 << (2 + 3) = 32
9 // NON (1 << 2) + 3 = 7 !
```

4 Structures de Contrôle

4.1 Conditionnelle if-else

Syntaxe if-else

if (condition) instruction; ou if (condition) {...} else {...}. La condition est vraie si non nulle.

Listing 21: If-else et dangling else

```
1 int a = 10, b = 20, c = 30;
2
3 // Bloc simple
4 if (a > b)
5     printf("a > b\n");
6
7 // Dangling else (associé au if le plus proche)
8 if (a > 5)
9     if (b > 15)
10        printf("Les deux\n");
11    else // Ce else est associé au deuxième if!
12        printf("Seulement b <= 15\n");
13
14 // Solution: accolades explicites
15 if (a > 5) {
16     if (b > 15)
17         printf("Les deux\n");
18 } else {
19     printf("a <= 5\n");
20 }
```

4.2 Conditionnelle switch

Syntaxe switch

`switch (expression) { case constante: ... break; default: ... }.` L'expression doit être de type entier. Le `break` est nécessaire pour éviter le fall-through.

Listing 22: Switch et fall-through

```
1 int day = 3;
2
3 switch (day) {
4     case 1:
5         printf("Lundi\n");
6         break;
7     case 2:
8         printf("Mardi\n");
9         break;
10    case 3:
11        printf("Mercredi\n");
12        // OUBLI de break: fall-through!
13    case 4:
14        printf("Jeudi\n"); // Exécuté aussi!
15        break;
16    default:
17        printf("Autre jour\n");
18 }
19
20 // Fall-through intentionnel
21 switch (day) {
22     case 1: case 2: case 3: case 4: case 5:
23         printf("Jour de semaine\n");
24         break;
25     case 6: case 7:
26         printf("Weekend\n");
27         break;
28 }
```

Limites du switch

Les étiquettes `case` doivent être des constantes de compilation. Pas de `case x`: si `x` est une variable. Pas de `case 1...5`: (extension gcc).

4.3 Boucle while

Syntaxe while

`while (condition) instruction;` La condition est évaluée AVANT chaque itération. Si fausse initialement, le corps ne s'exécute jamais.

Listing 23: While et condition

```
1 int i = 0;
2 while (i < 10) {
3     printf("%d ", i);
4     i++;
5 } // Affiche: 0 1 2 3 4 5 6 7 8 9
6
7 // Boucle infinie
8 while (1) {
9     // ...
10    if (condition)
11        break; // Sortie de la boucle
12 }
```

4.4 Boucle do-while

Syntaxe do-while

`do instruction; while (condition);` La condition est évaluée APRÈS chaque itération. Le corps s'exécute au moins une fois.

Listing 24: Do-while et exécution garantie

```
1 int i = 10;
2
```

```

3 // while: ne s'exécute pas
4 while (i < 10) {
5     printf("while: %d\n", i);
6 }
7
8 // do-while: s'exécute une fois
9 do {
10     printf("do-while: %d\n", i); // Affiche une fois
11 } while (i < 10);

```

4.5 Boucle for

Syntaxe for

for (initialisation; condition; incrémentation) instruction;. Équivalent à un while avec initialisation et incrémentation intégrées.

Listing 25: For et équivalence

```

1 // Forme classique
2 for (int i = 0; i < 10; i++) {
3     printf("%d ", i);
4 }
5
6 // Équivalent while
7 {
8     int i = 0;
9     while (i < 10) {
10         printf("%d ", i);
11         i++;
12     }
13 }
14
15 // Sections optionnelles
16 int i = 0;
17 for (; i < 10; ) { // Init et incr omises
18     printf("%d ", i);
19     i++;
20 }
21
22 // Boucle infinie
23 for (;;) { // Toutes sections omises
24     // ...
25 }

```

4.6 Instructions break et continue

Break et continue

break : sort de la boucle ou du switch englobant.
continue : passe à l'itération suivante de la boucle englobante.

Listing 26: Break et continue

```

1 // Break: sortie de boucle
2 for (int i = 0; i < 10; i++) {
3     if (i == 5)
4         break; // Sort de la boucle
5     printf("%d ", i);
6 } // Affiche: 0 1 2 3 4
7
8 // Continue: itération suivante
9 for (int i = 0; i < 10; i++) {
10     if (i % 2 == 0)
11         continue; // Passe aux nombres impairs
12     printf("%d ", i);
13 } // Affiche: 1 3 5 7 9
14
15 // Boucles imbriquées: break sort de la boucle IMMÉDIATE
16 for (int i = 0; i < 3; i++) {
17     for (int j = 0; j < 3; j++) {
18         if (j == 1)
19             break; // Sort de la boucle j, pas i
20         printf("(%d,%d) ", i, j);
21     }
22 } // Affiche: (0,0) (1,0) (2,0)

```

4.7 Instruction goto

Syntaxe goto

`goto étiquette;` suivi de `étiquette:` ailleurs dans la même fonction. Saut incondtionnel vers l'étiquette.

Usage de goto (Dijkstra)

Edsger W. Dijkstra, dans son article "Go To Statement Considered Harmful" (1968), a démontré que `goto` rend le code difficile à comprendre et à maintenir. L'utilisation de `goto` est généralement déconseillée en programmation moderne.

Cas acceptables (rares):

- Sortie de boucles profondément imbriquées
- Gestion d'erreurs avec nettoyage (pattern Linux kernel)

À éviter absolument:

- Sauts en arrière créant des boucles (utiliser `while`, `for`)
- Sauts complexes rendant le flux de contrôle illisible

Listing 27: Goto: sortie de boucles imbriquées

```
1 // Problème: break ne sort que d'une boucle
2 int found = 0;
3 for (int i = 0; i < 10 && !found; i++) {
4     for (int j = 0; j < 10; j++) {
5         if (condition(i, j)) {
6             found = 1;
7             break; // Sort seulement de la boucle j
8         }
9     }
10 }
11
12 // Solution avec goto (acceptable)
13 for (int i = 0; i < 10; i++) {
14     for (int j = 0; j < 10; j++) {
15         if (condition(i, j)) {
16             goto found; // Sort des deux boucles
17         }
18     }
19 }
20 found:
21 printf("Trouvé\n");
```

Listing 28: Goto: gestion d'erreurs (pattern kernel)

```
1 int process_data(void) {
2     int *buffer1 = malloc_int();
3     if (buffer1 == NULL)
4         goto error_buffer1;
5
6     int *buffer2 = malloc_int();
7     if (buffer2 == NULL)
8         goto error_buffer2;
9
10    int *buffer3 = malloc_int();
11    if (buffer3 == NULL)
12        goto error_buffer3;
13
14    // Traitement normal
15    process(buffer1, buffer2, buffer3);
16
17    // Nettoyage normal
18    free_int(buffer3);
19error_buffer3:
20    free_int(buffer2);
21error_buffer2:
22    free_int(buffer1);
23error_buffer1:
24    return -1; // Ou code d'erreur approprié
25 }
```

Listing 29: Transformation goto → boucles structurées

```
1 // Avec goto (à éviter: boucle déguisée)
2 int i = 0;
3 loop_start:
4     printf("%d ", i);
5     i++;
```

```

6     if (i < 10)
7         goto loop_start;
8
9     // Sans goto (préférable)
10    for (int i = 0; i < 10; i++) {
11        printf("%d ", i);
12    }
13
14    // Avec goto (acceptable: sortie multiple)
15    for (int i = 0; i < 100; i++) {
16        for (int j = 0; j < 100; j++) {
17            if (i * j > 500)
18                goto end; // Sortie propre
19        }
20    }
21    end:
22    printf("Terminé\n");
23
24    // Sans goto (avec flag, moins lisible)
25    int done = 0;
26    for (int i = 0; i < 100 && !done; i++) {
27        for (int j = 0; j < 100; j++) {
28            if (i * j > 500) {
29                done = 1;
30                break;
31            }
32        }
33    }
34    printf("Terminé\n");

```

Listing 30: Anti-patterns avec goto

```

1 // MAUVAIS: goto en arrière créant une boucle
2 int i = 0;
3 start:
4     printf("%d ", i);
5     i++;
6     if (i < 10)
7         goto start; // Utiliser for/while à la place!
8
9 // MAUVAIS: sauts croisés illisibles
10 if (condition1)
11     goto label2;
12 // code
13 label1:
14     // code
15     if (condition2)
16         goto label3;
17 label2:
18     // code
19     goto label1;
20 label3:
21     // Spaghetti code!
22
23 // MAUVAIS: saut dans un bloc
24 if (condition) {
25     // code
26 label: // Sauter ici depuis l'extérieur est dangereux
27     // code
28 }
29 goto label; // Variables locales non initialisées!

```

5 Fonctions

5.1 Déclaration et Définition

Déclaration vs définition

Déclaration : prototype de fonction (signature uniquement).

Définition : implémentation complète de la fonction.

Listing 31: Déclaration et définition

```

1 // Déclaration (prototype)
2 int add(int a, int b); // Noms des paramètres optionnels
3
4 // Définition
5 int add(int a, int b) {
6     return a + b;
7 }
8
9 // Déclaration sans prototype (K&R style, obsolète)
10 int multiply(); // Pas d'info sur les paramètres
11
12 // Appel
13 int x = add(5, 10);

```

5.2 Passage par Valeur

Passage par valeur

En C, les arguments sont TOUJOURS passés par valeur. Une copie de la valeur est transmise à la fonction. Modifications locales n'affectent pas l'appelant.

Listing 32: Passage par valeur

```
1 void increment(int x) {
2     x++; // Modifie la copie locale
3 }
4
5 int main() {
6     int a = 5;
7     increment(a);
8     printf("%d\n", a); // Affiche: 5 (inchangé)
9     return 0;
10 }
11
12 // Pour modifier: passer l'adresse (pointeur)
13 void increment_ptr(int *x) {
14     (*x)++;
15 }
16
17 int main() {
18     int a = 5;
19     increment_ptr(&a);
20     printf("%d\n", a); // Affiche: 6
21     return 0;
22 }
```

5.3 Valeur de Retour

Return

`return expression;` termine la fonction et retourne une valeur. Type de retour doit correspondre à la déclaration. `return;` pour fonctions `void`.

Listing 33: Valeurs de retour

```
1 int max(int a, int b) {
2     if (a > b)
3         return a;
4     return b; // Un seul return possible par exécution
5 }
6
7 // Fonction void
8 void print_max(int a, int b) {
9     printf("Max: %d\n", max(a, b));
10    return; // Optionnel pour void
11 }
12
13 // Return dans boucle
14 int find_first_positive(int arr[], int size) {
15     for (int i = 0; i < size; i++) {
16         if (arr[i] > 0)
17             return i; // Retour immédiat
18     }
19     return -1; // Non trouvé
20 }
```

5.4 Fonctions Récursives

Récursion

Une fonction peut s'appeler elle-même. Nécessite un cas de base pour éviter la récursion infinie. La pile d'appels stocke les contextes d'exécution.

Listing 34: Fonction récursive

```
1 // Factorielle
2 int factorial(int n) {
3     if (n <= 1) // Cas de base
4         return 1;
5     return n * factorial(n - 1); // Appel récursif
6 }
7
8 // Fibonacci
```

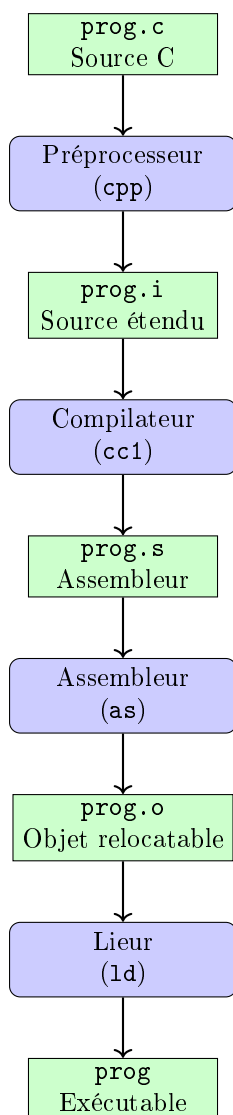
```

9  int fibonacci(int n) {
10     if (n <= 1)
11         return n;
12     return fibonacci(n - 1) + fibonacci(n - 2);
13 }
14
15 // Danger: récursion infinie
16 int infinite(int n) {
17     return infinite(n); // Stack overflow!
18 }

```

6 Compilation

6.1 Phases de Compilation



6.2 Préprocesseur

Directives du préprocesseur

#include, #define, #ifdef, #ifndef, #endif, #if, #else, #elif. Traitement textuel avant compilation.

Listing 35: Préprocesseur

```

1  #include <stdio.h> // Inclusion de header standard
2  #include "myheader.h" // Inclusion de header local
3
4  #define MAX 100 // Macro constante
5  #define SQUARE(x) ((x) * (x)) // Macro fonction
6
7  // Compilation conditionnelle
8  #ifndef DEBUG
9      printf("Debug mode\n");

```

```

10 #endif
11
12 // Afficher le résultat du préprocesseur
13 // gcc -E prog.c -o prog.i

```

6.3 Options de Compilation

Options gcc importantes

-Wall (warnings), -Wextra, -Werror, -std=c99, -O0/-O1/-O2/-O3 (optimisation), -g (debug), -S (assembleur), -c (objet).

Listing 36: Commandes gcc

```

1 # Compilation complète
2 gcc -Wall -Wextra -O2 -o prog prog.c
3
4 # Compilation séparée
5 gcc -c -Wall file1.c # Génère file1.o
6 gcc -c -Wall file2.c # Génère file2.o
7 gcc -o prog file1.o file2.o # Linkage
8
9 # Voir l'assembleur
10 gcc -S -O0 prog.c # Génère prog.s
11
12 # Debug avec gdb
13 gcc -g -o prog prog.c
14 gdb prog

```

7 Comportement Indéfini (UB)

7.1 Définition

Undefined Behavior

Comportement indéfini (UB) : le standard C ne spécifie PAS ce qui se passe. Le compilateur peut faire n'importe quoi : crash, résultat inattendu, code optimisé incorrectement, démons nasaux.

7.2 Exemples Classiques

Listing 37: UB courants

```

1 // 1. Division par zéro
2 int a = 5 / 0; // UB
3
4 // 2. Dépassement d'entier signé
5 int x = INT_MAX + 1; // UB (entiers signés)
6 unsigned int y = UINT_MAX + 1; // Défini: wrap around
7
8 // 3. Accès hors limites
9 int arr[10];
10 arr[15] = 42; // UB
11
12 // 4. Déréférencement de pointeur NULL
13 int *p = NULL;
14 *p = 42; // UB
15
16 // 5. Modification multiple sans sequence point
17 int i = 0;
18 i = i++ + ++i; // UB
19
20 // 6. Shift négatif ou >= largeur du type
21 int x = 1 << 32; // UB sur int 32 bits
22 int y = 1 << -1; // UB
23
24 // 7. Utilisation de variable non initialisée
25 int x;
26 printf("%d\n", x); // UB
27
28 // 8. Return sans valeur dans fonction non-void
29 int foo() {
30     // Oubli de return
31 } // UB si valeur utilisée

```

7.3 Séquence Points

Sequence Points

Points dans le programme où toutes les modifications précédentes sont garanties terminées. Exemples : `;`, `&&`, `||`, `?:`, `,` (virgule opérateur), appel de fonction.

Listing 38: Sequence points

```
1 int x = 5;
2
3 // OK: sequence point après ;
4 x = x + 1;
5 printf("%d\n", x);
6
7 // UB: x modifié deux fois sans sequence point
8 x = x++ + x++;
9
10 // OK: && est un sequence point
11 if (x++ > 5 && x < 10) { }
12
13 // UB: ordre d'évaluation non spécifié (pas de SP)
14 int y = foo() + bar(); // Quel ordre d'appel? Non défini
```

8 Bonnes Pratiques

Recommandations

- Utiliser `-Wall` `-Wextra` `-Werror` pour détecter les erreurs
- Initialiser toutes les variables
- Utiliser `unsigned` pour les manipulations de bits
- Éviter les modifications multiples dans une expression
- Utiliser des parenthèses pour clarifier la précedence
- Privilégier `for` pour les compteurs, `while` pour les conditions
- Toujours mettre `break` dans les `case` (sauf fall-through intentionnel)
- Utiliser des noms descriptifs pour les variables et fonctions